

第五章 二叉树 - 答案

1

一棵二叉树的先序、中序、后序序列如下，其中一部分未标出，请画出该二叉树。

- 先序序列：_ _ C D E _ G H I _ K
- 中序序列：C B _ _ F A _ J K I G
- 后序序列：_ E F D B _ J I H _ A

解答：

还原步骤。三个序列均有 11 位。后序末位给出根 A；中序以 A 划分后，左右子树各有 5 个结点。

- 左子树的后序为 _ E F D B，故根为 B。中序 C B _ _ F 表明 B 的左孩子为 C，右子树有 3 个结点。该右子树的先序、后序分别为 D E _ 和 E F D，故根为 D，两个孩子依次为 E、F。
- 右子树的先序为 G H I _ K，后序为 _ J I H _，故根为 G，其余结点全在左子树。依次结合中序 _ J K I G 可得：G 的左孩子为 H，H 的右孩子为 I，I 的左孩子为 J，J 的右孩子为 K。

还原出的二叉树：



逐序验证：

先序：A B C D E F G H I J K
中序：C B E D F A H J K I G
后序：C E F D B K J I H G A

将这三个完整序列中原题留空的位置重新替换为 _，恰好得到题目给出的三个序列。

2

证明：一棵二叉树的所有终端结点（叶结点），在前序序列、中序序列、后序序列中都按相同的相对位置出现。

解答：

采用对节点数 n 的数学归纳法。

基础情况：

- 当 $n = 0$ （空二叉树）时，前/中/后序序列均为空，命题平凡成立。
- 当 $n = 1$ （只有一个节点 A 的二叉树）时，前/中/后序序列都只是 A，叶节点就是 A，相对位置显然相同。命题成立。

归纳假设：假设当节点数 $< n$ ($n \geq 2$) 时，对任意二叉树，叶节点在前/中/后序序列中的相对位置都相同。

归纳步：考虑节点数为 n 的二叉树 T ，设根节点为 r ，左子树 T_L （节点数 $n_1 < n$ ），右子树 T_R （节点数 $n_2 < n$ ， $n_1 + n_2 = n - 1$ ）。记 T_L 的前/中/后序序列分别为 $L_{\text{pre}}, L_{\text{in}}, L_{\text{post}}$ ， T_R 的类似记为 $R_{\text{pre}}, R_{\text{in}}, R_{\text{post}}$ 。

则 T 的前/中/后序序列可以表示为：

$$T_{\text{pre}} = r L_{\text{pre}} R_{\text{pre}} \quad (1)$$

$$T_{\text{in}} = L_{\text{in}} r R_{\text{in}} \quad (2)$$

$$T_{\text{post}} = L_{\text{post}} R_{\text{post}} r \quad (3)$$

由于 r 是根节点（不是叶节点，叶节点都在 T_L 或 T_R 中），叶节点仅来自 T_L 和 T_R 。

由归纳假设， T_L 的叶节点在 $L_{\text{pre}}, L_{\text{in}}, L_{\text{post}}$ 中的相对位置相同； T_R 的叶节点在 $R_{\text{pre}}, R_{\text{in}}, R_{\text{post}}$ 中的相对位置也相同。

由于在每种序列中 L 和 R 都是连续的子串（ T_{pre} 中 L_{pre} 在前， R_{pre} 在后； T_{in} 中 L_{in} 在前， R_{in} 在后； T_{post} 中 L_{post} 在前， R_{post} 在后），所以：

- 在 T_{pre} 中，叶节点的出现顺序 = (T_L 的叶节点按 L_{pre} 顺序) + (T_R 的叶节点按 R_{pre} 顺序)。
- 在 T_{in} 中，叶节点的出现顺序 = (T_L 的叶节点按 L_{in} 顺序) + (T_R 的叶节点按 R_{in} 顺序)。
- 在 T_{post} 中，叶节点的出现顺序 = (T_L 的叶节点按 L_{post} 顺序) + (T_R 的叶节点按 R_{post} 顺序)。

由于每段内相对位置相同，且三序列中"先 L 后 R "的拼接顺序一致，所以 T 中叶节点在三种序列中的相对位置也相同。归纳完成。■

3

(1) 哈夫曼树有 199 个结点，求叶子结点数

解答：

哈夫曼树是**严格二叉树**（没有度为 1 的结点），只有度为 0（叶子）和度为 2（内部）的结点。设叶子数为 n_0 ，度为 2 的结点数为 n_2 。

两个关系：

- 总结点数： $n_0 + n_2 = 199$ 。
- 每个内部结点恰有两条指向孩子的边，而全树的边数为结点数减一，因此 $2n_2 = 199 - 1 = 198$ 。

由两式解得 $n_2 = 99$ ， $n_0 = 100$ 。

结论：叶子结点数为 100。

简化版：哈夫曼树每次合并两棵子树，新增 1 个内部结点，减少 1 个"待合并子树的根"（净增 1 个新节点）。从 n 个叶子出发，最终结点数 = $2n - 1$ 。所以 $2n - 1 = 199 \Rightarrow n = 100$ 。

(2) 设计哈夫曼编码

解答：

将题目中 1 至 8 号字母的频率放大 100 倍便于构造（{7, 19, 2, 6, 32, 3, 21, 10}）。

构造过程（每次取两个最小合并）：

步骤	队列状态	合并
初始	2, 3, 6, 7, 10, 19, 21, 32	—
1	5, 6, 7, 10, 19, 21, 32	2 + 3 = 5
2	7, 10, 11, 19, 21, 32	5 + 6 = 11
3	11, 17, 19, 21, 32	7 + 10 = 17
4	19, 21, 28, 32	11 + 17 = 28
5	28, 32, 40	19 + 21 = 40
6	40, 60	28 + 32 = 60
7	100	40 + 60 = 100

哈夫曼树（左 0、右 1）：



哈夫曼编码表：

字母	频率	哈夫曼编码	二进制编码
1	0.07	1100	000
2	0.19	00	001
3	0.02	11110	010
4	0.06	1110	011
5	0.32	10	100
6	0.03	11111	101
7	0.21	01	110
8	0.10	1101	111

(3) 两种编码方案比较

解答：

哈夫曼编码的平均码长：

$$L_H = \sum p_i \cdot l_i = 0.07 \times 4 + 0.19 \times 2 + 0.02 \times 5 + 0.06 \times 4 + 0.32 \times 2 + 0.03 \times 5 + 0.21 \times 2 + 0.10 \times 4 \quad (4)$$

$$= 0.28 + 0.38 + 0.10 + 0.24 + 0.64 + 0.15 + 0.42 + 0.40 = \boxed{2.61} \quad (5)$$

二进制等长编码（8 个字母用 3 位）的平均码长：

$$L_B = 3 \quad (6)$$

比较：

维度	哈夫曼编码	二进制等长编码
平均码长	2.61 位	3.00 位
存储 1 万字符所需 bit	26100	30000
高频字符的码长	短（2 位）	恒为 3 位
低频字符的码长	长（4-5 位）	恒为 3 位
实现复杂度	需编码表/树	无需查表
解码方式	按哈夫曼树走	固定切分（每 3 位一组）
唯一可译性	是（前缀码）	是（定长）

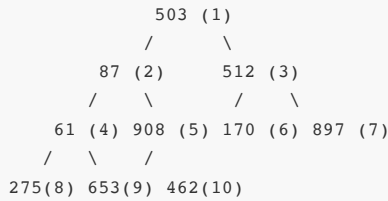
结论：哈夫曼编码优于二进制等长编码。在字符出现频率分布不均匀的场合，哈夫曼编码通过对高频字符分配短码、低频字符分配长码，能显著降低平均码长（这里是 3.00 → 2.61，约节省 13% 的空间）。代价是：编码长度不均匀，解码需借助哈夫曼树或编码表，实现稍复杂；编码表本身也要随字符集一起存储，对极短消息反而不划算。

4

(1) 建堆（最小堆）

解答：

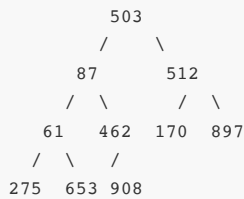
将序列视为层序存储的完全二叉树（数组下标 1-based）：



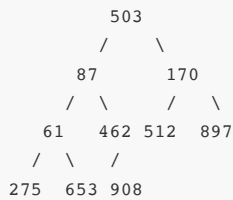
初始完全二叉树（上图即为第一步的树）。

从最后一个非叶子结点 ($\lfloor 10/2 \rfloor = 5$ ，即 908) 开始，依次向上做 sift-down：

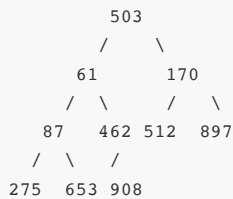
1. sift-down(5, 908): 与子 462 交换 ($462 < 908$)。



2. sift-down(4, 61): 子 275, 653，最小 275 > 61，不交换。
3. sift-down(3, 512): 子 170, 897，最小 170 < 512，交换到位置 3，再看 512 的子（已无更小子）。

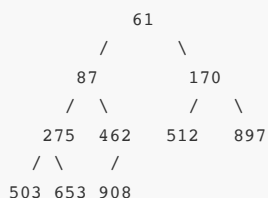


4. sift-down(2, 87): 子 61, 462，最小 61 < 87，交换。



5. sift-down(1, 503): 子 61, 170，最小 61 < 503，交换。
 - 503 到位置 2: 子 87, 462，最小 87 < 503，交换。
 - 503 到位置 4: 子 275, 653，最小 275 < 503，交换。
 - 503 到位置 8: 无子，停止。

最终堆：



对应的层序数组：`[61, 87, 170, 275, 462, 512, 897, 503, 653, 908]`

最小元素 `61` 已经在堆顶，可以直接输出。

(2) 最小堆的最大值元素位置

解答：

至少有一个最大值元素位于叶子结点；若最大值唯一，它只能位于叶子结点。

设最大值为 M 。若某个值为 M 的结点不是叶子，由最小堆性质，它的每个孩子都不小于 M ，因而也等于 M 。沿孩子向下，最终必能到达一个值为 M 的叶子。

若允许相同关键码，最大值也可能在内部结点：例如所有结点的关键码都相同，根结点也是最大值。因此批改时不能把“最大值只能在叶子”作为无条件结论。

(3) 初始建堆的时间代价

解答：

结论：最坏比较次数 $T(n) < 2n$ ，因此自底向上建堆为 $O(n)$ 。实际比较次数依赖输入序列，不能只由 n 给出一个固定的等式。

推导。以某结点为根的子树高度记为该结点到最后代代的边数。一次向下调整每考察一层孩子，至多比较两次关键码：先在两个孩子中选较小者，再与父结点比较；只有一个孩子时比较次数更少。最多考察与该结点高度相同的层数，因此比较次数至多为其高度的两倍。

在层序编号为 `1..n` 的完全二叉树中，高度至少为 j 的结点数为 $\lfloor n/2^j \rfloor$ 。将每个结点的高度按层累加，可得

$$T(n) \leq 2 \sum_{j \geq 1} \left\lfloor \frac{n}{2^j} \right\rfloor < 2 \sum_{j \geq 1} \frac{n}{2^j} = 2n. \quad (7)$$

这给出题目要求的线性上界；它不是每个输入的精确比较次数。

5

构造最小加权路径长度的 k 叉树 ($n \geq 1, k \geq 2$, 权值非负)

解答：

满 k 叉树的必要条件。一棵每个内部结点都有恰好 k 个孩子的满 k 叉树，若有 n 个叶子结点，则内部结点数为 $\frac{n-1}{k-1}$ 。这要求 $(n-1)$ 能被 $(k-1)$ 整除，即：

$$n \equiv 1 \pmod{k-1} \quad (8)$$

否则需要补“虚权值”（权值为 0 的虚拟叶节点）。补的个数为：

$$\text{虚权值数} = (k-1) - ((n-1) \bmod (k-1)) \quad (9)$$

（如果余数已经是 0，则不需要补。）

构造算法（与哈夫曼树类似）：

1. 把 n 个权值看作 n 棵只有根节点的 k 叉树。
2. 如果需要加虚权值，则第一次合并时从原始数据中取 $(k - \text{虚权值数})$ 个，加上 虚权值数 个 0，构成 k 叉树，根节点权值为这 k 个权值之和；将这棵新树加入集合。
3. 之后每次从集合中取 k 个权值最小的树作为子树，合并成新的 k 叉树，新根的权值为这 k 个子树根权值之和，从集合中删除这 k 棵子树，将新 k 叉树加入集合。
4. 重复步骤 3，直到集合中只剩一棵树——这棵树就是最优 k 叉树。

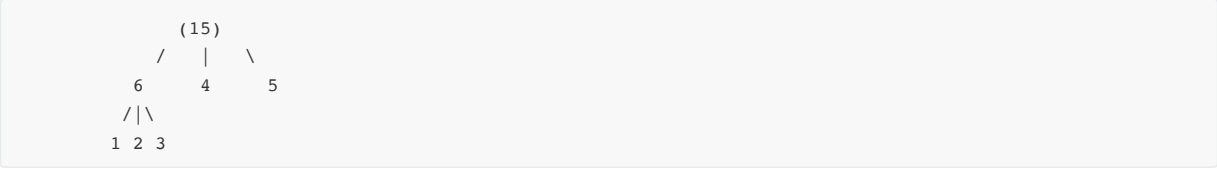
示例：构造最优 3 叉树，权值集合 $\{1, 2, 3, 4, 5\}$

$n = 5, k = 3, k - 1 = 2, n \bmod 2 = 1$ 。满足 $n \equiv 1 \pmod 2$ ，不需要补虚权值。

构造过程：

步骤	队列	合并
初始	1, 2, 3, 4, 5	—
1	6, 4, 5	$1 + 2 + 3 = 6$
2	15	$6 + 4 + 5 = 15$

最优 3 叉树：



最小加权路径长度：

$$WPL = 1 \cdot 2 + 2 \cdot 2 + 3 \cdot 2 + 4 \cdot 1 + 5 \cdot 1 = 2 + 4 + 6 + 4 + 5 = \boxed{21} \tag{10}$$

若题目给定的权值集 n 为偶数（例如 $\{1, 2, 3, 4\}$ ），则需要补 1 个虚权值 0，第一次合并变成 $0 + 1 + 2 = 3$ ，之后再 $3 + 3 + 4 = 10$ ，得到的最小 $WPL = 1 \cdot 2 + 2 \cdot 2 + 3 \cdot 1 + 4 \cdot 1 = 2 + 4 + 3 + 4 = 13$ 。